MIT VNAV 2023 - Lab 3 Individual Solutions

Quaternion APIs and Quadrotor Modelling and Control

Name: Wenqian Zhang Student ID: 24020036087

Deliverable 1: Transformations in Practice

The required concise answers are also provided in deliverable_1.txt.

Task	C++ expression
geometry_msgs -> existing tf2 quaternion	tf2::fromMsg(quat_msg, quat_tf);
tf2 -> newly declared geometry_msgs quaternion	geometry_msgs::Quaternion quat_msg = tf2::toMsg(quat_tf);
Scalar component	double w = quat_tf.getW();
Yaw	double yaw = tf2::getYaw(quat_t);
ROS quaternion -> Eigen rotation matrix	tf::quaternionMsgToEigen(quat_msg, quat_eigen); then quat_eigen.normalized().toRotationMatrix();

Deliverable 2A: Rank of the Quadrotor Allocation Matrix

Let rotor i produce force $c_f\omega_i^2 e_3$ and reaction torque $s_i c_d \omega_i^2 e_3$, where s_i is +1 or -1. For rotor position $\rho_i = (x_i, y_i, 0)^T$, $\rho_i \times c_f\omega_i^2 e_3 = (c_f y_i, -c_f x_i, 0)^T \omega_i^2$.

Use the X geometry shown in the assignment, with arm length l and rotor positions

$$\rho_1 = (l/\sqrt{2})[1,1,0]^T, \quad \rho_2 = (l/\sqrt{2})[-1,1,0]^T,$$

$$\rho_3 = (l/\sqrt{2})[-1,-1,0]^T, \quad \rho_4 = (l/\sqrt{2})[1,-1,0]^T.$$

Set $a = c_f l/\sqrt{2}$ and $d = c_d$. The two zero lateral-force rows of the full 6-by-4 F matrix do not affect rank, so it is sufficient to display the rows for $(f_z, \tau_x, \tau_y, \tau_z)$.

Functional quadrotor (a)

The spin signs are (+,-,+,-), so

c_f	c_f	c_f	c_f
a	a	-a	-a
-a	a	a	-a
d	-d	d	-d

For nonzero c_f, a, and d, these four rows are independent. Therefore rank(F_a) = 4: total thrust and all three body moments can be chosen independently.

Modified quadrotor (b)

Reversing rotors 3 and 4 changes the spin signs to (+,-,-,+), giving

c_f	c_f	c_f	c_f
a	a	-a	-a
-a	a	a	-a
d	-d	-d	d

The yaw row is exactly -d/a times the pitch row. The other three rows remain independent, so rank(F_b) = 3.

> Result. Quadrotor (b) cannot command independent pitch and yaw moments. It therefore cannot in general track all four outputs [x,y,z,Ψ].

Deliverable 2B: Explicit Drag Compensation

The empirical body-frame model in the question is

$$^{b}F_{drag} = -D\,[v_x|v_x|,\ v_y|v_y|,\ v_z|v_z|]^{T},$$

$$D = \mathrm{diag}(0.1,\ 0.1,\ 0.2).$$

This sign convention makes each component oppose its velocity. If $^{w}R_b$ maps body coordinates to the world frame, then

$$^{w}F_{drag} = {}^{w}R_b\ {}^{b}F_{drag}.$$

A standard translational part of the geometric controller forms the nominal desired force

$$F_{nom} = -K_x e_x - K_v e_v + mg\,e_3 + m\,a_d.$$

Because the true dynamics contain the additional drag force, add an equal and opposite feed-forward term:

$$F_{cmd} = F_{nom} - {}^{w}F_{drag}.$$

The controller then constructs the desired attitude using the compensated thrust direction

$$b_{3d} = F_{cmd} / ||F_{cmd}||,$$

together with the desired yaw direction, and commands the collective thrust by projecting F_{cmd} onto the current body thrust axis. If the course convention defines thrust along $-e_3$, the corresponding signs are reversed consistently.

Practical implementation

Step	Operation
1	Estimate world velocity and rotate it into the body frame: $^{b}v = {}^{b}R_w\ {}^{w}v$.
2	Evaluate the componentwise quadratic drag model with D = diag(0.1,0.1,0.2).
3	Rotate the predicted drag back to the world frame.
4	Subtract the predicted drag from the nominal desired force before constructing desired attitude and thrust.

> Result. The model-based term cancels the predictable aerodynamic disturbance, reducing steady tracking error at speed while the feedback terms handle residual model error.